A beginner's introduction to C#

Europe Connexion's team*

February 25, 2026

Abstract

This document is meant to help beginners and intermediates learn how to write code in C# using the .NET framework. C# is a modern, object-oriented programming language developed by Microsoft that makes it easy and intuitive to build robust applications. C# is versatile and powerful, making it a great way to learn the fundamentals of programming and builds a strong foundation for learning other languages in the future. By the end of this document the reader should have a solid grasp on the syntax of the language, as well as how to work with the .NET framework and create various types of applications.

Contents

1 Introduction **5**
1.1 What is C#? . . . 5
1.2 Why Learn C#? . . . 5
1.3 The Foundation of C# . . . 5
1.3.1 Object-Oriented Programming . . . 5
1.3.2 Type Safety . . . 6
1.3.3 The .NET Framework . . . 6

2 Getting Started **6**
2.1 Installing Visual Studio . . . 6
2.1.1 Downloading Visual Studio . . . 6
2.1.2 Installation Process . . . 6
2.2 Creating Your First Project . . . 7
2.2.1 Starting a New Project . . . 7
2.2.2 Configuring Your Project . . . 7
2.3 Running Your First Program . . . 7
2.3.1 Understanding the Code . . . 8
2.3.2 Running the Program . . . 8
2.4 Understanding the Visual Studio Interface . . . 8
2.5 Common Keyboard Shortcuts . . . 9

3 Basic Syntax **9**
3.1 Variables and Types . . . 9
3.1.1 What is a Variable? . . . 9
3.1.2 Why Are Variables Important? . . . 9
3.1.3 Data Types in C# . . . 10
3.1.4 Declaring and Initializing Variables . . . 10
3.1.5 The var Keyword . . . 11
3.1.6 Type Conversion . . . 11
3.2 Arrays and Collections . . . 11
3.2.1 What is an Array? . . . 11
3.2.2 Declaring and Initializing Arrays . . . 11
3.2.3 Accessing Array Elements . . . 12
3.2.4 Array Properties and Methods . . . 12
3.2.5 Multi-Dimensional Arrays . . . 12
3.3 Control Flow . . . 12
3.3.1 What is Conditional Execution? . . . 12
3.3.2 If Statements . . . 12
3.3.3 Else If Statements . . . 13
3.3.4 Comparison Operators . . . 13
3.3.5 Logical Operators . . . 14
3.3.6 Switch Statements . . . 14
3.4 Loops . . . 15
3.4.1 For Loops . . . 15

3.4.2 While Loops . . . 15
3.4.3 Do-While Loops . . . 16
3.4.4 Foreach Loops . . . 16
3.4.5 Loop Control Statements . . . 16
3.5 Methods and Functions . . . 16
3.5.1 What is a Method? . . . 16
3.5.2 Method Structure . . . 17
3.5.3 Creating and Calling Methods . . . 17
3.5.4 Method Parameters . . . 17
3.5.5 Method Overloading . . . 18
3.6 Comments and Formatting . . . 18
3.6.1 Single-Line Comments . . . 18
3.6.2 Multi-Line Comments . . . 18
3.6.3 XML Documentation Comments . . . 19
3.7 Common Operators . . . 19
3.7.1 Arithmetic Operators . . . 19
3.7.2 Assignment Operators . . . 19
3.7.3 Increment and Decrement . . . 20
3.8 Coding Practice: Basic Syntax . . . 20

4 Object-Oriented Programming 20
4.1 Classes and Objects . . . 20
4.2 Encapsulation and Access Modifiers . . . 21
4.3 Inheritance and Polymorphism . . . 21
4.4 Interfaces . . . 21
4.5 Best Practices . . . 21
4.6 Coding Practice: Object-Oriented Programming . . . 22

5 Mini Projects 22
5.1 Project 1: Calculator Application . . . 22
5.2 Project 2: To-Do List Manager . . . 22
5.3 Project 3: Number Guessing Game . . . 23
5.4 Project 4: Contact Book . . . 23
5.5 Project 5: Grade Calculator . . . 23

6 Managing Your Projects with GitHub 23
6.1 What is Git and GitHub? . . . 24
6.2 Installing Git . . . 24
6.3 Creating a GitHub Account . . . 24
6.4 Basic Git Commands . . . 24
6.5 Workflow for C# Projects . . . 25
6.6 Using Git in Visual Studio . . . 25
6.7 Best Practices . . . 26

7 Conclusion **26**
7.1 What You've Learned 26
7.2 Next Steps in Your C# Journey 27
7.3 Resources for Continued Learning 27
7.4 Tips for Success 28
7.5 Final Thoughts 28

1 Introduction

1.1 What is C#?

C# (pronounced "C Sharp") is a modern, object-oriented programming language developed by Microsoft in 2000 as part of its .NET initiative. It was designed by Anders Hejlsberg and his team to be simple, powerful, and type-safe. C# combines the computing power of C++ with the programming ease of Visual Basic, making it an ideal language for both beginners and experienced developers.

C# is widely used for developing desktop applications, web applications, mobile apps, games (especially with Unity), and cloud-based services. It runs on the .NET framework, which provides a vast library of pre-built functionality that developers can leverage to build applications quickly and efficiently.

1.2 Why Learn C#?

Learning C# offers numerous advantages for aspiring programmers:

- **Versatility:** C# can be used to build virtually any type of application, from desktop software to mobile apps and web services.
- **Industry Demand:** C# developers are in high demand across various industries, particularly in enterprise software development.
- **Strong Foundation:** Learning C# teaches fundamental programming concepts that transfer well to other languages like Java, C++, and JavaScript.
- **Rich Ecosystem:** The .NET framework provides extensive libraries, tools, and community support.
- **Modern Features:** C# continuously evolves with modern programming features like async/await, LINQ, pattern matching, and more.
- **Game Development:** C# is the primary language for Unity, one of the most popular game development engines.

1.3 The Foundation of C#

C# is built on several key principles that make it powerful and accessible:

1.3.1 Object-Oriented Programming

C# is fundamentally object-oriented, meaning it organizes code into objects that contain both data and behavior. This approach makes code more modular, reusable, and easier to maintain. The four pillars of OOP in C# are:

- **Encapsulation:** Bundling data and methods that operate on that data within a single unit (class)

- **Inheritance:** Creating new classes based on existing ones
- **Polymorphism:** The ability of objects to take multiple forms
- **Abstraction:** Hiding complex implementation details and showing only necessary features

1.3.2 Type Safety

C# is a strongly-typed language, which means every variable must have a defined type. This helps catch errors at compile-time rather than runtime, making your programs more reliable and easier to debug.

1.3.3 The .NET Framework

The .NET framework is a software development platform that provides:

- Common Language Runtime (CLR) for executing programs
- Base Class Library (BCL) with thousands of pre-built classes
- Tools for debugging, testing, and deployment

2 Getting Started

2.1 Installing Visual Studio

To begin programming in C#, you'll need an Integrated Development Environment (IDE). Visual Studio is Microsoft's flagship IDE and the most popular choice for C# development.

2.1.1 Downloading Visual Studio

1. Visit the official Visual Studio website: `https://visualstudio.microsoft.com/`
2. Choose Visual Studio Community (free for students and individual developers)
3. Download the installer for your operating system
4. Run the installer once the download is complete

2.1.2 Installation Process

When you run the Visual Studio installer, you'll be presented with various workload options:

- **.NET desktop development:** For building Windows desktop applications

- **ASP.NET and web development:** For creating web applications
- **Mobile development with .NET:** For building mobile apps
- **Game development with Unity:** For creating games

For beginners, we recommend starting with ".NET desktop development" which includes everything needed to build console and desktop applications.

2.2 Creating Your First Project

Once Visual Studio is installed, you're ready to create your first C# project.

2.2.1 Starting a New Project

1. Open Visual Studio
2. Click on "Create a new project"
3. In the search box, type "Console App"
4. Select "Console App (.NET Core)" or "Console App (.NET Framework)"
5. Click "Next"

2.2.2 Configuring Your Project

After selecting the project template:

1. Enter a project name (e.g., "MyFirstCSharpApp")
2. Choose a location to save your project
3. Optionally create a solution name
4. Click "Create"

Visual Studio will generate a basic project structure with a `Program.cs` file containing a simple "Hello World" program.

2.3 Running Your First Program

Your first C# program is automatically generated when you create a console application. Here's what the code looks like:

```
using System;

namespace MyFirstCSharpApp
{
    class Program
    {
```

```
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");
        }
    }
}
```

2.3.1 Understanding the Code

Let's break down what each part means:

- `using System;` - Imports the System namespace, which contains fundamental classes
- `namespace MyFirstCSharpApp` - Organizes your code into a logical group
- `class Program` - Defines a class named Program
- `static void Main(string[] args)` - The entry point of your application
- `Console.WriteLine("Hello World!");` - Prints text to the console

2.3.2 Running the Program

To execute your program:

1. Press F5 or click the "Start" button (green arrow) in the toolbar
2. Visual Studio will compile and run your program
3. A console window will appear displaying "Hello World!"
4. Press any key to close the console window

2.4 Understanding the Visual Studio Interface

Visual Studio provides a powerful development environment with several key components:

- **Solution Explorer:** Shows the structure of your project and all its files
- **Code Editor:** Where you write your code, with syntax highlighting and IntelliSense
- **Output Window:** Displays build messages and debugging information
- **Error List:** Shows compilation errors and warnings
- **Properties Window:** Displays and allows editing of selected item properties
- **Toolbar:** Quick access to common commands like Build and Run

2.5 Common Keyboard Shortcuts

Learning keyboard shortcuts will significantly improve your productivity:

- **F5:** Start debugging
- **Ctrl + F5:** Start without debugging
- **Ctrl + S:** Save current file
- **Ctrl + Shift + B:** Build solution
- **Ctrl + K, Ctrl + C:** Comment selected lines
- **Ctrl + K, Ctrl + U:** Uncomment selected lines
- **F12:** Go to definition
- **Ctrl + Space:** Trigger IntelliSense

3 Basic Syntax

This section covers the foundational syntax elements of C# that every beginner should know.

3.1 Variables and Types

3.1.1 What is a Variable?

A variable is a named storage location in memory that holds a value. Think of it as a labeled box where you can store data that your program needs to remember and use later. Every variable in C# has:

- A **name** (identifier) that you use to refer to it
- A **type** that determines what kind of data it can hold
- A **value** that is stored in it

3.1.2 Why Are Variables Important?

Variables are fundamental to programming because they allow you to:

- Store and reuse data throughout your program
- Make calculations and manipulate data
- Keep track of program state
- Make your code more readable and maintainable
- Avoid repeating the same values throughout your code

3.1.3 Data Types in C#

C# is a statically typed language: each variable has a type known at compile time. Common built-in types include:

Numeric Types:

- `int` – integer numbers from -2,147,483,648 to 2,147,483,647 (e.g., 0, -3, 42)
- `long` – larger integers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
- `float` – single-precision floating point (e.g., 3.14f)
- `double` – double-precision floating point numbers (e.g., 3.14159)
- `decimal` – high-precision decimal for financial calculations (e.g., 19.99m)

Text Types:

- `char` – single character (e.g., 'A', 'z', '7')
- `string` – sequences of characters (text) (e.g., "Hello World")

Boolean Type:

- `bool` – boolean values: `true` or `false`

3.1.4 Declaring and Initializing Variables

To create a variable, you must declare its type and give it a name. You can also initialize it with a value:

```
// Declaration and initialization
int count = 10;
double pi = 3.14159;
bool isReady = true;
string name = "Alice";
char grade = 'A';

// Declaration without initialization
int x;
x = 5; // assignment

// Multiple declarations
int a = 1, b = 2, c = 3;
```

3.1.5 The var Keyword

C# allows you to use the `var` keyword for implicit typing. The compiler infers the type from the value:

```
var number = 42;        // inferred as int
var message = "Hello";  // inferred as string
var price = 19.99;      // inferred as double
```

Note: You must initialize a `var` variable when you declare it.

3.1.6 Type Conversion

Sometimes you need to convert values from one type to another:

Implicit Conversion (automatic):

```
int myInt = 9;
double myDouble = myInt;  // automatic casting: int to double
```

Explicit Conversion (manual):

```
double myDouble = 9.78;
int myInt = (int)myDouble;  // manual casting: double to int (loses .78)

// Using Convert class
string numText = "123";
int number = Convert.ToInt32(numText);
```

3.2 Arrays and Collections

3.2.1 What is an Array?

An array is a data structure that can store multiple values of the same type in a single variable. Instead of declaring separate variables for each value, you can store them all in one array.

3.2.2 Declaring and Initializing Arrays

```
// Declaration and initialization with values
int[] numbers = { 1, 2, 3, 4, 5 };
string[] names = { "Alice", "Bob", "Charlie" };

// Declaration with size, then assign values
int[] scores = new int[5];
scores[0] = 95;
scores[1] = 87;
scores[2] = 92;

// Declaration with size and initialization
int[] data = new int[3] { 10, 20, 30 };
```

3.2.3 Accessing Array Elements

Array elements are accessed using zero-based indexing:

```
int[] numbers = { 10, 20, 30, 40, 50 };

Console.WriteLine(numbers[0]);  // prints 10 (first element)
Console.WriteLine(numbers[2]);  // prints 30 (third element)

numbers[1] = 25;  // change second element to 25
```

3.2.4 Array Properties and Methods

```
int[] numbers = { 5, 2, 8, 1, 9 };

int length = numbers.Length;        // get array length (5)
Array.Sort(numbers);                // sort array: {1, 2, 5, 8, 9}
Array.Reverse(numbers);             // reverse array
int index = Array.IndexOf(numbers, 8);  // find index of value 8
```

3.2.5 Multi-Dimensional Arrays

C# supports multi-dimensional arrays for representing matrices or grids:

```
// 2D array (matrix)
int[,] matrix = new int[3, 3]
{
    { 1, 2, 3 },
    { 4, 5, 6 },
    { 7, 8, 9 }
};

Console.WriteLine(matrix[1, 2]);  // prints 6 (row 1, column 2)
```

3.3 Control Flow

3.3.1 What is Conditional Execution?

Conditional execution allows your program to make decisions and execute different code based on certain conditions. This is essential for creating programs that respond to different situations.

3.3.2 If Statements

The `if` statement executes code only if a condition is true:

```
int age = 18;

if (age >= 18)
```

```
{
    Console.WriteLine("You are an adult.");
}

// if-else
int score = 75;
if (score >= 60)
{
    Console.WriteLine("You passed!");
}
else
{
    Console.WriteLine("You failed.");
}
```

3.3.3 Else If Statements

Use `else if` to test multiple conditions:

```
int score = 85;

if (score >= 90)
{
    Console.WriteLine("Grade: A");
}
else if (score >= 80)
{
    Console.WriteLine("Grade: B");
}
else if (score >= 70)
{
    Console.WriteLine("Grade: C");
}
else if (score >= 60)
{
    Console.WriteLine("Grade: D");
}
else
{
    Console.WriteLine("Grade: F");
}
```

3.3.4 Comparison Operators

Operators used in conditions:

- `==` – equal to

- `!=` – not equal to
- `>` – greater than
- `<` – less than
- `>=` – greater than or equal to
- `<=` – less than or equal to

3.3.5 Logical Operators

Combine multiple conditions:

- `&&` – AND (both conditions must be true)
- `||` – OR (at least one condition must be true)
- `!` – NOT (inverts the condition)

Example:

```
int age = 25;
bool hasLicense = true;

if (age >= 18 && hasLicense)
{
    Console.WriteLine("You can drive.");
}

bool isWeekend = true;
bool isHoliday = false;

if (isWeekend || isHoliday)
{
    Console.WriteLine("No work today!");
}
```

3.3.6 Switch Statements

The `switch` statement is useful for multi-way branching based on a single value:

```
int day = 3;

switch (day)
{
    case 1:
        Console.WriteLine("Monday");
        break;
    case 2:
```

```
        Console.WriteLine("Tuesday");
        break;
    case 3:
        Console.WriteLine("Wednesday");
        break;
    default:
        Console.WriteLine("Other day");
        break;
}
```

3.4 Loops

3.4.1 For Loops

The `for` loop is used when you know how many times you want to repeat code:

```
// Basic for loop
for (int i = 0; i < 5; i++)
{
    Console.WriteLine("Count: " + i);
}
// Output: 0, 1, 2, 3, 4

// Loop through an array
int[] numbers = { 10, 20, 30, 40, 50 };
for (int i = 0; i < numbers.Length; i++)
{
    Console.WriteLine(numbers[i]);
}
```

3.4.2 While Loops

The `while` loop repeats code as long as a condition is true:

```
int count = 0;
while (count < 5)
{
    Console.WriteLine("Count: " + count);
    count++;
}

// While loop with user input
string input = "";
while (input != "quit")
{
    Console.WriteLine("Enter 'quit' to exit:");
    input = Console.ReadLine();
}
```

3.4.3 Do-While Loops

The `do-while` loop executes code at least once, then repeats while a condition is true:

```
int number;
do
{
    Console.WriteLine("Enter a positive number:");
    number = Convert.ToInt32(Console.ReadLine());
}
while (number <= 0);
```

3.4.4 Foreach Loops

The `foreach` loop is ideal for iterating through arrays and collections:

```
string[] fruits = { "Apple", "Banana", "Orange" };

foreach (string fruit in fruits)
{
    Console.WriteLine(fruit);
}
```

3.4.5 Loop Control Statements

- `break` – exits the loop immediately
- `continue` – skips the current iteration and moves to the next one

```
for (int i = 0; i < 10; i++)
{
    if (i == 5)
        break;  // exit loop when i is 5

    if (i % 2 == 0)
        continue;  // skip even numbers

    Console.WriteLine(i);  // prints only odd numbers: 1, 3
}
```

3.5 Methods and Functions

3.5.1 What is a Method?

A method is a block of code that performs a specific task. Methods help organize code, make it reusable, and improve readability. In C#, methods are always part of a class.

3.5.2 Method Structure

A method consists of:

- **Access modifier:** `public`, `private`, etc.
- **Return type:** The type of value returned (or `void` for no return)
- **Method name:** Following naming conventions (PascalCase)
- **Parameters:** Input values (optional)
- **Method body:** The code to execute

3.5.3 Creating and Calling Methods

```
// Method with no parameters and no return value
static void SayHello()
{
    Console.WriteLine("Hello!");
}

// Method with parameters
static void Greet(string name)
{
    Console.WriteLine("Hello, " + name + "!");
}

// Method with return value
static int Add(int a, int b)
{
    return a + b;
}

// Calling methods
SayHello();              // Output: Hello!
Greet("Alice");          // Output: Hello, Alice!
int result = Add(5, 3);  // result = 8
```

3.5.4 Method Parameters

Parameters can be passed in different ways:

```
// Multiple parameters
static void DisplayInfo(string name, int age, bool isStudent)
{
    Console.WriteLine($"{name} is {age} years old.");
    Console.WriteLine($"Student status: {isStudent}");
}
```

```
// Optional parameters with default values
static void CreateUser(string name, int age = 18, string country = "USA")
{
    Console.WriteLine($"{name}, {age}, from {country}");
}

CreateUser("Alice");              // Uses default age and country
CreateUser("Bob", 25);            // Uses default country
CreateUser("Charlie", 30, "UK");  // All parameters specified
```

3.5.5 Method Overloading

You can create multiple methods with the same name but different parameters:

```
static int Multiply(int a, int b)
{
    return a * b;
}

static double Multiply(double a, double b)
{
    return a * b;
}

static int Multiply(int a, int b, int c)
{
    return a * b * c;
}

int result1 = Multiply(3, 4);         // calls first method
double result2 = Multiply(3.5, 2.0);  // calls second method
int result3 = Multiply(2, 3, 4);      // calls third method
```

3.6 Comments and Formatting

3.6.1 Single-Line Comments

Use `//` for single-line comments:

```
// This is a single-line comment
int x = 5;  // Comment after code
```

3.6.2 Multi-Line Comments

Use `/* ... */` for longer explanations:

```
/*
This is a multi-line comment.
It can span multiple lines.
Use it for detailed explanations.
*/
int y = 10;
```

3.6.3 XML Documentation Comments

Use `///` for documenting methods and classes:

```
/// <summary>
/// Calculates the sum of two integers.
/// </summary>
/// <param name="a">The first integer</param>
/// <param name="b">The second integer</param>
/// <returns>The sum of a and b</returns>
static int Add(int a, int b)
{
    return a + b;
}
```

3.7 Common Operators

3.7.1 Arithmetic Operators

- `+` – Addition
- `-` – Subtraction
- `*` – Multiplication
- `/` – Division
- `%` – Modulus (remainder)

3.7.2 Assignment Operators

- `=` – Assign
- `+=` – Add and assign
- `-=` – Subtract and assign
- `*=` – Multiply and assign
- `/=` – Divide and assign

3.7.3 Increment and Decrement

```
int x = 5;
x++;  // increment (x = 6)
x--;  // decrement (x = 5)

int a = 5;
int b = ++a;  // pre-increment: a=6, b=6
int c = a++;  // post-increment: a=7, c=6
```

3.8 Coding Practice: Basic Syntax

Try these exercises to reinforce what you've learned:

1. Create variables of different types and print them to the console.
2. Write a program that converts temperatures from Celsius to Fahrenheit using the formula: F = C * 9/5 + 32.
3. Create an array of your favorite movies and use a loop to print each one.
4. Write a method that takes two numbers and returns the larger one.
5. Create a program that asks the user for their age and prints whether they are a child (¡ 13), teenager (13-19), adult (20-64), or senior (65+).
6. Write a program that calculates the sum of all numbers from 1 to 100 using a loop.
7. Create a method that takes an array of integers and returns the average value.
8. Write a program that uses a switch statement to print the name of a month based on a number (1-12).

4 Object-Oriented Programming

This section expands on the object-oriented features of C#, including how to define and use classes, inheritance, and interfaces.

4.1 Classes and Objects

A class is a blueprint for objects. It can contain fields, properties, methods and constructors.

```
class Person
{
    // field
    private string name;
```

```
    // property
    public int Age { get; set; }

    // constructor
    public Person(string name, int age)
    {
        this.name = name;
        Age = age;
    }

    // method
    public void Greet()
    {
        Console.WriteLine($"Hello, my name is {name} and I'm {Age}.");
    }
}
```

4.2 Encapsulation and Access Modifiers

Use `private`, `protected`, `internal`, and `public` to control access. Prefer properties for exposing state rather than public fields.

4.3 Inheritance and Polymorphism

Classes can inherit from a single base class and override virtual methods to provide polymorphic behavior.

```
class Animal { public virtual void Speak() => Console.WriteLine("..."); }
class Dog : Animal { public override void Speak() => Console.WriteLine("Woof"); }

Animal a = new Dog();
a.Speak(); // prints "Woof" because of polymorphism
```

4.4 Interfaces

Interfaces declare a contract without implementation. A class can implement multiple interfaces.

```
interface ILogger { void Log(string message); }
class ConsoleLogger : ILogger { public void Log(string message) => Console.WriteLine(message
```

4.5 Best Practices

Keep classes small and focused, prefer composition over inheritance when appropriate, and use interfaces to express contracts for testability.

4.6 Coding Practice: Object-Oriented Programming

Try these exercises to practice OOP concepts:

1. Create a `BankAccount` class with properties for account number, balance, and owner name. Add methods to deposit, withdraw, and check balance.
2. Design a `Vehicle` base class and derive `Car` and `Motorcycle` classes from it. Override a `DisplayInfo()` method in each derived class.
3. Create an interface `IShape` with a method `CalculateArea()`. Implement it in `Circle`, `Rectangle`, and `Triangle` classes.
4. Build a `Student` class with properties for name, ID, and a list of grades. Add methods to add grades, calculate average, and determine if the student is passing (average ¿= 60).
5. Create a class hierarchy for employees: a base `Employee` class and derived classes `Manager`, `Developer`, and `Intern` with different salary calculation methods.

5 Mini Projects

Apply what you've learned by building these small projects:

5.1 Project 1: Calculator Application

Build a console calculator that:

- Prompts the user to enter two numbers
- Asks for an operation (+, -, *, /)
- Displays the result
- Continues until the user chooses to exit
- Handles division by zero and invalid input

5.2 Project 2: To-Do List Manager

Create a simple to-do list application that:

- Allows users to add tasks
- Displays all tasks with numbers
- Lets users mark tasks as complete
- Allows users to delete tasks
- Saves tasks to a text file

5.3 Project 3: Number Guessing Game

Develop a guessing game where:

- The program generates a random number between 1 and 100
- The user has multiple attempts to guess it
- The program provides hints (too high or too low)
- It tracks the number of attempts
- It offers to play again when finished

5.4 Project 4: Contact Book

Build a contact management system that:

- Stores contacts with name, phone, and email
- Allows adding, viewing, searching, and deleting contacts
- Uses a `Contact` class to represent each contact
- Uses a `List<Contact>` to store all contacts
- Provides a menu-driven interface

5.5 Project 5: Grade Calculator

Create a grade calculation tool that:

- Takes multiple student names and their scores
- Calculates average, highest, and lowest scores
- Assigns letter grades (A, B, C, D, F)
- Displays a summary report
- Uses arrays and loops effectively

6 Managing Your Projects with GitHub

Learning version control is essential for modern software development.

6.1 What is Git and GitHub?

Git is a distributed version control system that tracks changes in your code. **GitHub** is a web-based platform that hosts Git repositories and provides collaboration features.

Benefits of using Git and GitHub:

- Track changes in your code over time
- Revert to previous versions if something breaks
- Collaborate with other developers
- Backup your code in the cloud
- Showcase your projects to potential employers

6.2 Installing Git

1. Visit `https://git-scm.com/downloads`
2. Download the installer for your operating system
3. Run the installer with default settings
4. Verify installation by opening a terminal and typing: `git --version`

6.3 Creating a GitHub Account

1. Go to `https://github.com`
2. Click "Sign up"
3. Follow the registration process
4. Verify your email address

6.4 Basic Git Commands

Essential commands for working with Git:

```
# Configure your identity
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"

# Initialize a new repository
git init

# Check status of files
git status
```

```
# Stage files for commit
git add filename.cs
git add .  # stage all files

# Commit changes
git commit -m "Description of changes"

# View commit history
git log

# Connect to remote repository
git remote add origin https://github.com/username/repository.git

# Push changes to GitHub
git push origin main

# Pull changes from GitHub
git pull origin main
```

6.5 Workflow for C# Projects

1. Create a new repository on GitHub
2. Clone it to your local machine or initialize Git in your project folder
3. Add your C# project files (make sure to add a `.gitignore` for C#)
4. Stage and commit your changes regularly
5. Push your commits to GitHub
6. Continue this cycle as you develop

6.6 Using Git in Visual Studio

Visual Studio has built-in Git support:

- Use the Git menu to clone repositories
- View changes in the Git Changes window
- Commit and push directly from Visual Studio
- Create and switch between branches
- View commit history

6.7 Best Practices

- Commit frequently with clear, descriptive messages
- Use a `.gitignore` file to exclude build files and dependencies
- Write a good README.md to describe your project
- Use branches for new features
- Keep your main branch stable and working

7 Conclusion

Congratulations on completing this introduction to C#! You've learned the fundamental concepts that form the foundation of programming in C# and the .NET framework. Let's recap what you've accomplished:

7.1 What You've Learned

Throughout this document, you've gained knowledge in:

- **C# Fundamentals:** Understanding what C# is, its history, and why it's a valuable language to learn
- **Development Environment:** Setting up Visual Studio and navigating its interface
- **Basic Syntax:** Working with variables, data types, operators, and type conversions
- **Control Structures:** Using conditionals, loops, and decision-making constructs
- **Arrays and Collections:** Storing and manipulating multiple values efficiently
- **Methods:** Creating reusable code blocks with parameters and return values
- **Object-Oriented Programming:** Designing classes, using inheritance, implementing interfaces, and applying encapsulation
- **Version Control:** Managing your code with Git and GitHub

7.2 Next Steps in Your C# Journey

Now that you have a solid foundation, here are recommended areas to explore next:

1. **Advanced OOP Concepts:** Explore abstract classes, static members, generics, and design patterns
2. **Exception Handling:** Learn to handle errors gracefully using try-catch blocks
3. **File I/O:** Work with reading from and writing to files
4. **LINQ:** Master Language Integrated Query for powerful data manipulation
5. **Asynchronous Programming:** Understand async/await for responsive applications
6. **Database Integration:** Connect to databases using ADO.NET or Entity Framework
7. **Web Development:** Build web applications with ASP.NET Core
8. **Mobile Development:** Create mobile apps with Xamarin or .NET MAUI
9. **Game Development:** Start building games with Unity
10. **Testing:** Write unit tests to ensure code quality

7.3 Resources for Continued Learning

To continue your C# education, consider these resources:

- **Microsoft Learn:** Free official tutorials and learning paths (`docs.microsoft.com/learn`)
- **C# Documentation:** Comprehensive reference (`docs.microsoft.com/dotnet/csharp/`)
- **Practice Platforms:** LeetCode, HackerRank, Codewars for coding challenges
- **YouTube Channels:** IAmTimCorey, Brackeys, dotnet for video tutorials
- **Books:** "C# Player's Guide" by RB Whitaker, "Head First C#" by Andrew Stellman
- **Community:** Stack Overflow, Reddit r/csharp, C# Discord servers

7.4 Tips for Success

As you continue learning, keep these tips in mind:

- **Practice Regularly:** Code every day, even if just for 30 minutes
- **Build Projects:** Apply what you learn by creating real applications
- **Read Other's Code:** Study open-source projects on GitHub
- **Debug Thoughtfully:** Use breakpoints and the debugger to understand your code
- **Don't Fear Errors:** Errors are learning opportunities, not failures
- **Join Communities:** Connect with other learners and experienced developers
- **Stay Current:** Follow C# updates and new features
- **Teach Others:** Explaining concepts solidifies your understanding

7.5 Final Thoughts

Learning to program is a journey, not a destination. C# is a powerful and versatile language that will serve you well whether you're building desktop applications, web services, mobile apps, or games. The concepts you've learned here—variables, control flow, functions, and object-oriented programming—are universal and will transfer to other languages and technologies you encounter.

Remember that every expert programmer was once a beginner. Be patient with yourself, celebrate small victories, and enjoy the process of creating software. The skills you're building now will open doors to exciting career opportunities and enable you to bring your ideas to life through code.

Keep coding, keep learning, and most importantly, have fun building amazing things with C#!

**This document was created by the Europe Connexion's team to help aspiring programmers begin their journey with C#. We hope it has been helpful in your learning process. Happy coding!*